

Technical Report Workbook

Project Information

Project No.

Project Title

Prepared By

Project Start Date

Completion Date

Repository / Project Link

DOCUMENTATION GUIDANCE

Use this workbook while the project is in progress. Record meaningful design decisions, testing, errors, corrections, and lessons as they happen. Complete the final summary only after the project has been tested and reviewed.

Professional writing tip: Write what you actually did. Use precise verbs such as implemented, tested, identified, corrected, validated, documented, and refactored.

1. Project Context and Security Need

Define why the work exists before describing how you built it.

WHEN TO UPDATE Draft early, then revise after final testing so the description matches the completed project.

Project Overview

WHAT TO WRITE: Explain what the project is, what it is intended to do, who or what would use it, and what result or output it produces.

HOW TO WRITE IT: Write 3-5 concise sentences. Lead with the purpose, then describe the input, process, and output at a high level.

Problem / Security Need

WHAT TO WRITE: Describe the security problem, operational need, vulnerability, control gap, or learning objective that motivated the project.

HOW TO WRITE IT: State the risk or need first. Then explain why a technical solution, control, or validation process is appropriate.

2. Requirements, Scope, and Policy

Keep written requirements aligned with what the solution actually enforces.

WHEN TO UPDATE Update whenever requirements, scoring rules, scope boundaries, or expected behavior change.

Functional Requirements

WHAT TO WRITE: List the actions the project must perform or the outcomes it must produce.

HOW TO WRITE IT: Use short requirement statements beginning with verbs such as Accept, Validate, Detect, Calculate, Classify, Log, Alert, or Display.

Security Policy / Decision Rules

WHAT TO WRITE: Document thresholds, validation criteria, allowed or denied behavior, severity levels, or any rules the solution applies.

HOW TO WRITE IT: Write exact rules. If the logic changes during development, update this section so the report and implementation stay consistent.

3. Technical Design and Approach

Explain the solution structure without copying the entire source code.

WHEN TO UPDATE Add to this as major phases are completed. This becomes the technical approach section of your final report.

Program / System Flow

WHAT TO WRITE: Describe the order of operations from input through processing, validation, decision logic, storage, output, or response.

HOW TO WRITE IT: Write chronologically. Mention major stages and how information moves between them.

Tools, Languages, Libraries, and Key Concepts

WHAT TO WRITE: Record the technologies used and explain briefly what each one contributes to the project.

HOW TO WRITE IT: Do more than list names. Add a short phrase explaining the role of each tool, library, command, framework, or programming concept.

4. Running Development Log

Capture meaningful progress while you work so you do not have to reconstruct it later.

Use short entries for completed tasks, important design decisions, feature changes, and verification steps. You do not need to document every line of code.

Development Entry 1

Date / Phase		Task / Change	
Why / Result			

Development Entry 2

Date / Phase		Task / Change	
Why / Result			

Development Entry 3

Date / Phase		Task / Change	
Why / Result			

5. Troubleshooting and Issue Resolution

Record real errors, unexpected behavior, causes, corrections, and verification.

WHEN TO UPDATE Fill this out whenever the project behaves differently than expected. Real mistakes are useful evidence of troubleshooting ability.

Issue 1 - What Happened?

WHAT TO WRITE: Record the error, unexpected behavior, affected component, and what you expected to happen instead.

HOW TO WRITE IT: Be specific. Include only the relevant error name or short message rather than pasting long console output.

Issue 1 - Cause, Correction, and Verification

WHAT TO WRITE: Explain the root cause, what you changed, and how you proved the correction worked.

HOW TO WRITE IT: Use a clear sequence: issue -> cause -> change -> retest -> result.

5. Troubleshooting and Issue Resolution

Continued

Issue 2 - What Happened?

WHAT TO WRITE: Record another meaningful error, unexpected behavior, or failed test.

HOW TO WRITE IT: State what you observed, what you expected, and where the issue occurred.

Issue 2 - Cause, Correction, and Verification

WHAT TO WRITE: Explain the root cause, correction, and retest outcome.

HOW TO WRITE IT: Keep the sequence clear: issue -> cause -> change -> verification.

6. Testing Plan and Results

Use planned test cases to verify expected behavior and edge conditions.

Use synthetic or non-sensitive test data. Do not place real passwords, tokens, credentials, or confidential information in test records or screenshots.

Test Case TC-01

Purpose / Scenario

Expected Result

Actual Result

Pass / Fail

Notes

Test Case TC-02

Purpose / Scenario

Expected Result

Actual Result

Pass / Fail

Notes

Test Case TC-03

Purpose / Scenario

Expected Result

Actual Result

Pass / Fail

Notes

Test Case TC-04

Purpose / Scenario

Expected Result

Actual Result

Pass / Fail

Notes

7. Results and Findings

Write what testing actually demonstrated, not what you expected to happen.

WHEN TO UPDATE Complete after core requirements are implemented and testing is substantially finished.

Implementation Results

WHAT TO WRITE: Summarize which requirements were successfully implemented and what the finished solution produced.

HOW TO WRITE IT: Use past tense and evidence-based wording such as implemented, verified, produced, detected, classified, blocked, or passed.

Key Findings

WHAT TO WRITE: Record 2-4 important observations from testing, including behavior, edge cases, weaknesses, or noteworthy outcomes.

HOW TO WRITE IT: A finding should explain both what you observed and why it matters.

8. Security Considerations and Limitations

Show scope awareness and avoid overstating what the solution can do.

WHEN TO UPDATE Complete after testing and before the final summary.

Security Considerations

WHAT TO WRITE: Document safe data handling, sensitive information concerns, access assumptions, logging implications, or other precautions relevant to the project.

HOW TO WRITE IT: Describe practical safeguards and boundaries. Keep claims tied to what the project actually handles.

Limitations

WHAT TO WRITE: Explain what the solution does not evaluate, prevent, detect, automate, or guarantee.

HOW TO WRITE IT: State limitations plainly. Clear scope boundaries make technical reports more credible.

9. Lessons Learned and Future Improvements

Use real development experience rather than generic cybersecurity phrases.

WHEN TO UPDATE Add lessons during development, then refine the wording at the end. Future improvements should be realistic next steps.

Lessons Learned

WHAT TO WRITE: Record specific programming, security, debugging, testing, documentation, or design lessons from the project.

HOW TO WRITE IT: Use concrete examples: what was confusing, what you learned, and how your approach changed.

Future Improvements

WHAT TO WRITE: List realistic enhancements for a later version after current project requirements are complete.

HOW TO WRITE IT: Separate unfinished required work from optional future enhancements.

10. Final Professional Summary

Write this last. It should stand alone as a concise technical handoff.

WHEN TO UPDATE Complete only after final testing, cleanup, and review.

Executive / Technical Summary

WHAT TO WRITE: In one concise paragraph, state the security need, what you built, the technical approach, testing outcome, and overall result.

HOW TO WRITE IT: Aim for 5-8 sentences. Focus on purpose, implementation, validation, and outcome rather than narrating every coding step.

Recommendation / Next Step

WHAT TO WRITE: State what should happen next if this were reviewed by a supervisor or internal technical team.

HOW TO WRITE IT: Recommend review, expanded testing, deployment controls, integration, maintenance, or further development based on demonstrated results and limitations.

11. Repository and Evidence Checklist

Use this before publishing or submitting the project.

Mark each item before publishing or submitting the repository.

- ☐ README reflects the final behavior and scope.
- ☐ Source code runs without known errors.
- ☐ Test cases include expected and actual results.
- ☐ Screenshots use synthetic or non-sensitive data only.
- ☐ No credentials, API keys, tokens, or personal secrets are included.
- ☐ Temporary debug output is removed or intentionally documented.
- ☐ Documentation folder contains project instructions and/or report materials.
- ☐ Lessons Learned and Future Improvements reflect the actual project experience.
- ☐ File and folder names are clear and consistent.
- ☐ Final report has been reviewed for spelling, grammar, and technical accuracy.

Final Repository / Submission Notes